What Is Deep Learning?

DSC 330 Applied Machine Learning

April 18, 2025

Unit: Deep Learning #1

The Plan

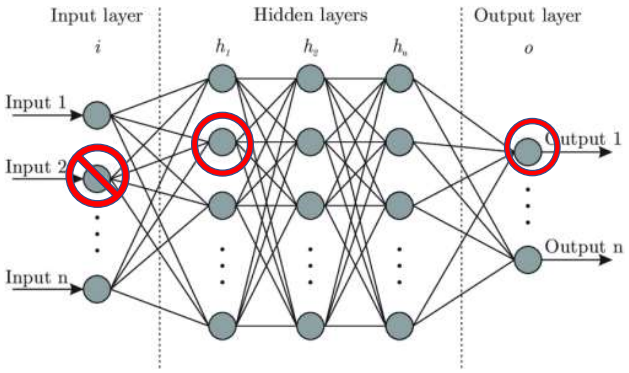
- Revisit perceptron
  - What is it? How does it work?
- Feed-forward neural networks
  - Basic architecture. How does it work?
- How does deep learning differ?
- Preview of future topics
  - Training (incl. backpropagation), CNNs, RNNs, GANs
- Tools
  - TensorFlow and Keras

Artificial Neural Net

Overview

Neural net overview

- ANNs composed of **units**
  - Neurons
  - Nodes in network
- Perceptron we saw earlier
- Inputs (several)
- Output (1, but copied)
- Function
  - Weights on inputs + bias
  - Activation function



Bre, Gimenez, and Fachinotti, Prediction of Wind Pressure Coefficients
https://www.researchgate.net/figure/Artificial-neural-network-architecture-ANN-i-h-1-h-2-h-n-o_fig1_321259051

The Unit (Perceptron)

One unit (neuron or cell) of an artificial neural network

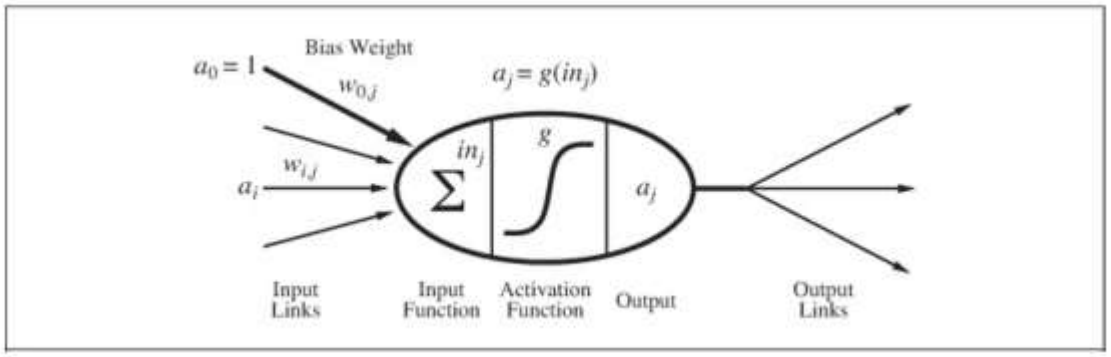
Unit (Perceptron)

- Inputs: $\mathbf{a}$ incl. $a_0 \equiv 1$
- Each feature multiplied by weight $\mathbf{w}$ incl. bias w_0
- Transform output with **activation function** (e.g., Heaviside, sigmoid):

$$g\left(\sum_{j=0}^{m} a_j \cdot w_j\right)$$

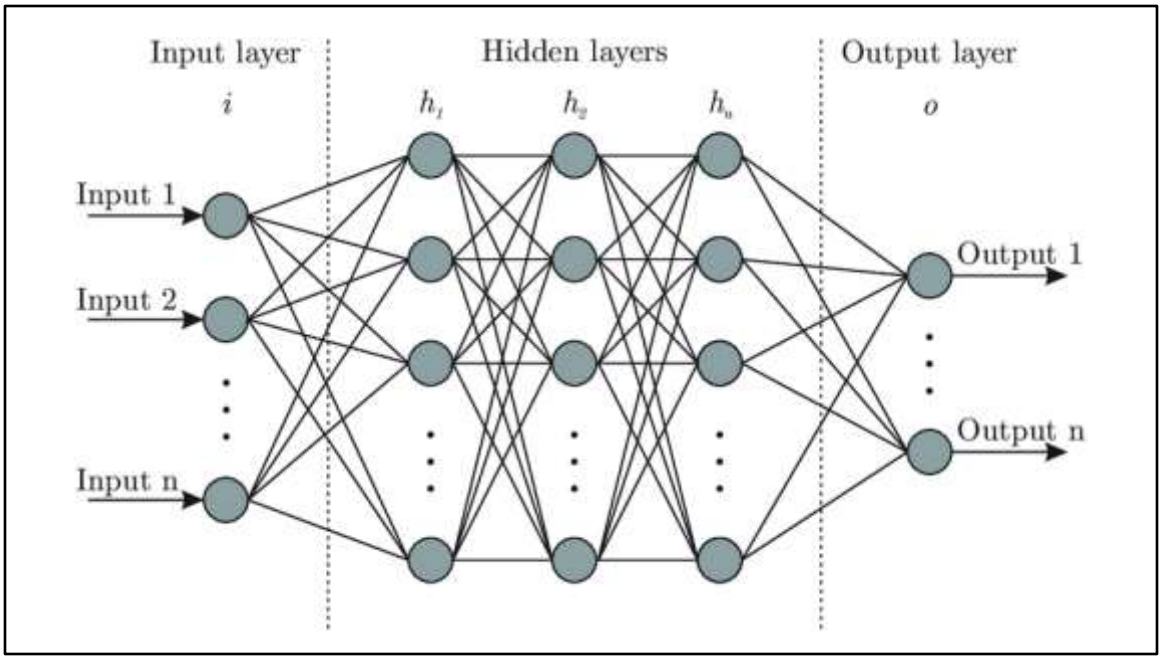
- Perceptron learning algorithm (originally hand-tuned)
- Linear model

Unit: How it works



Unit: How it works

- **Multiply** each **input** by its corresponding **weight**
- Take the **sum**
- Also add the **bias**
- Apply an **activation function** (many choices) to the sum
- **Output** the result
  - If **output layer** unit, then this is your **class label** or **predicted value**
  - If **hidden layer**, then feed into units in the next layer (next slide …)
    - Known as forward propagation


Input layer
Hidden layers
Output layer
i
h_1
h_2
h_n
o
Input 1
Input 2
Input n
Output 1
Output n

Activation Functions

"Does the neuron fire?"

What is an Activation Function?

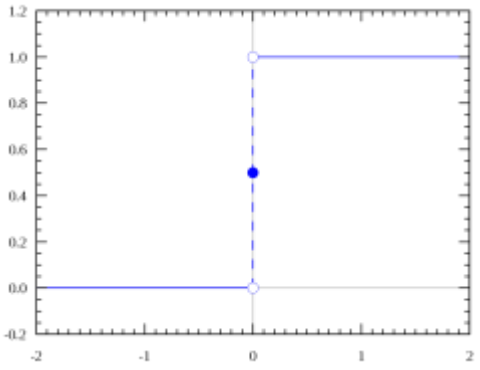
- Attached to each unit in network (except input layer)
- Applied after computing dot product

$$g\left(\sum_{j=0}^{m} x_j \cdot w_j\right)$$

- Determines whether the neuron "**fires**" (propagates forward)
- **"Is the input to this particular unit relevant to the model's prediction?"**
- Normalizes output to some value in a range, e.g., [0, 1] or [-1, 1]
- Many choices! **Important hyperparameter!**

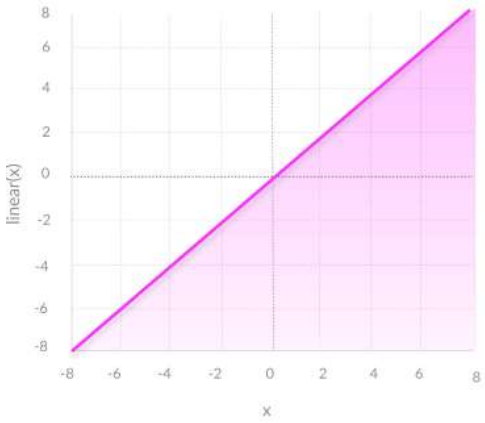
Choices for Activation Functions

- Heaviside (binary) step function
  - Not differentiable at $z = 0$
  - Everywhere else, derivative is 0
  - Backpropagation needs derivative
  - So can't train in the usual way ☹



Choices for Activation Functions

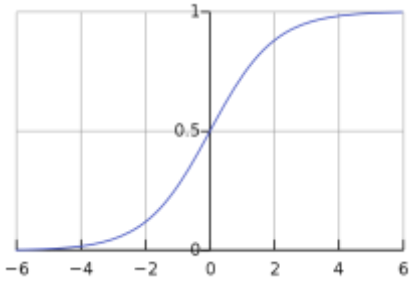
- Heaviside (binary) step function
- Linear function, e.g., $f(z) = z$
  - Note: No activation function is a special case of this
  - Derivative is a constant
  - So again, can't use backpropagation ☹
  - Also, all layers collapse into one!
  - So we can only learn linear functions! ☹



Choices for Activation Functions

- Heaviside (binary) step function
- Linear function, e.g., $f(z) = z$
- Sigmoid (logistic) function

$$f(z) = \frac{1}{1 + e^{-z}}$$

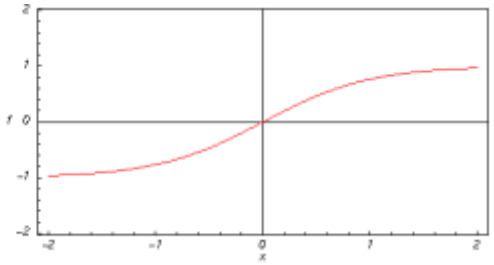


Advantages:
- Smooth gradient
- Output in range [0,1]
- Clear predictions

Disadvantages:
- Vanishing gradient (not enough info)
- Outputs not 0 centered
- Computationally expensive

Choices for Activation Functions

- Heaviside (binary) step function
- Linear function
- Sigmoid (logistic) function
- Tanh (hyperbolic tangent)

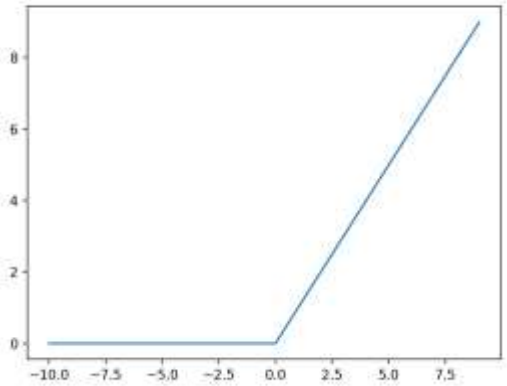


Advantages:
- Zero centered
- Otherwise like sigmoid

Disadvantages:
- Same as sigmoid

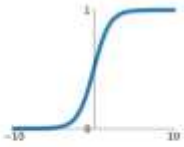
Choices for Activation Functions

- Heaviside (binary) step function
- Linear function
- Sigmoid (logistic) function
- Tanh (hyperbolic tangent)
- ReLU (rectified linear unit)
  - Computationally efficient
  - Non-linear, works with backprop.
  - But "dying ReLU problem" ☹
- Today there are many choices …



Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



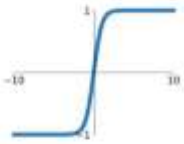
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$



Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

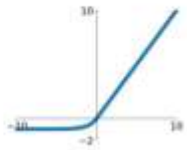
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



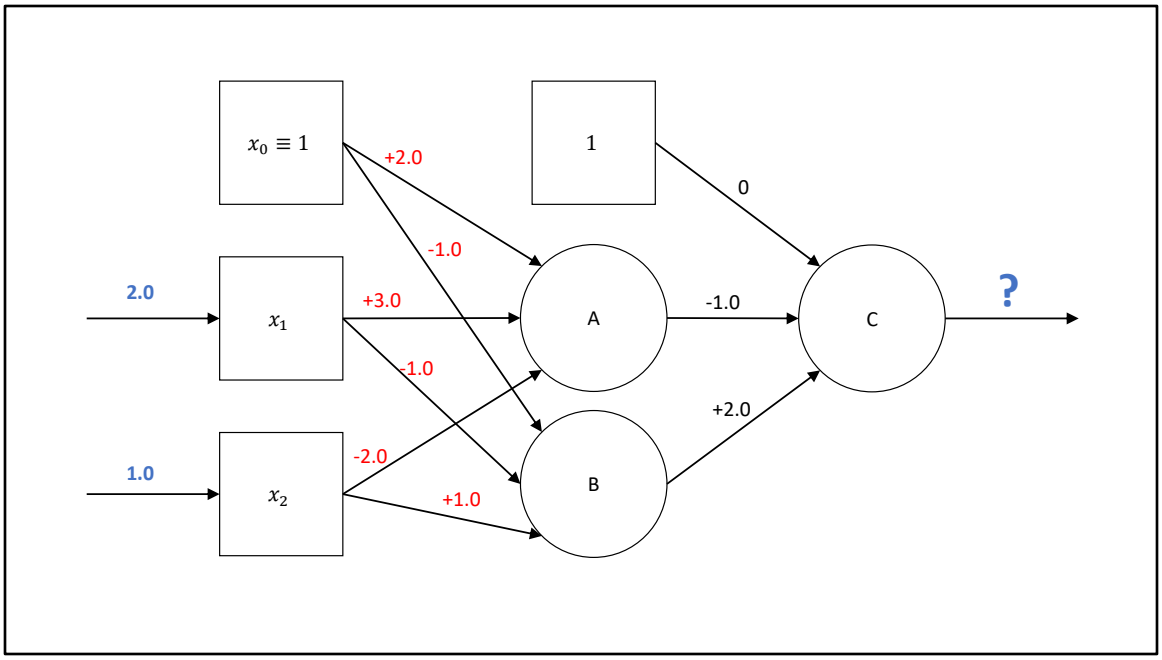
https://towardsdatascience.com/complete-guide-of-activation-functions-34076e95d044

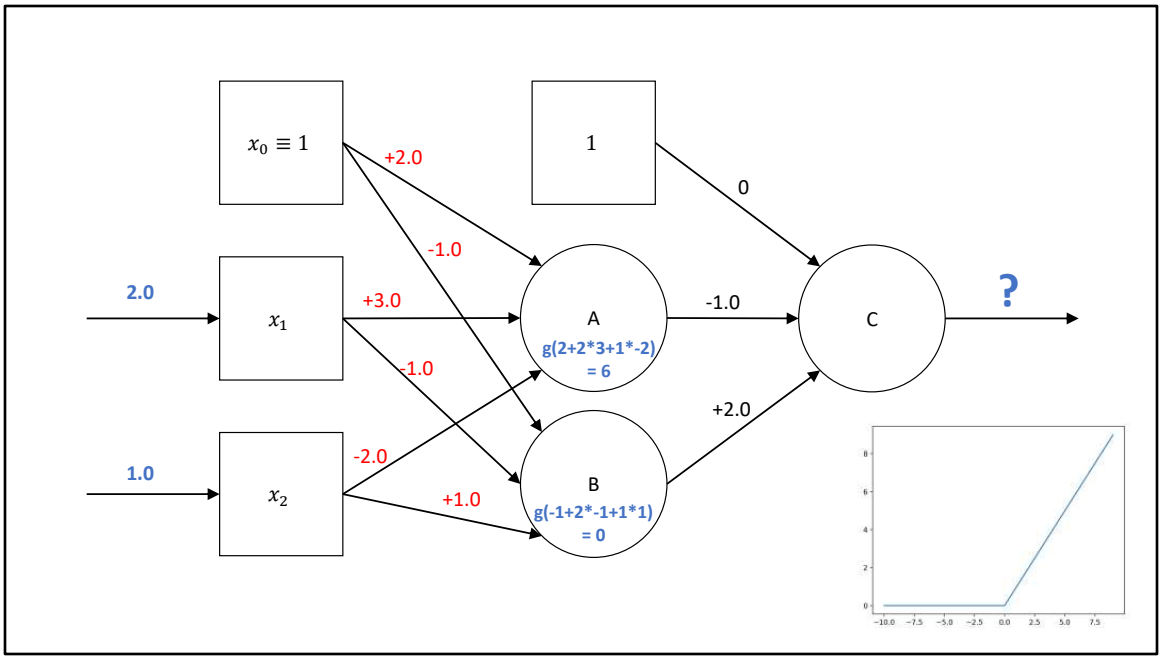
Feedforward Networks

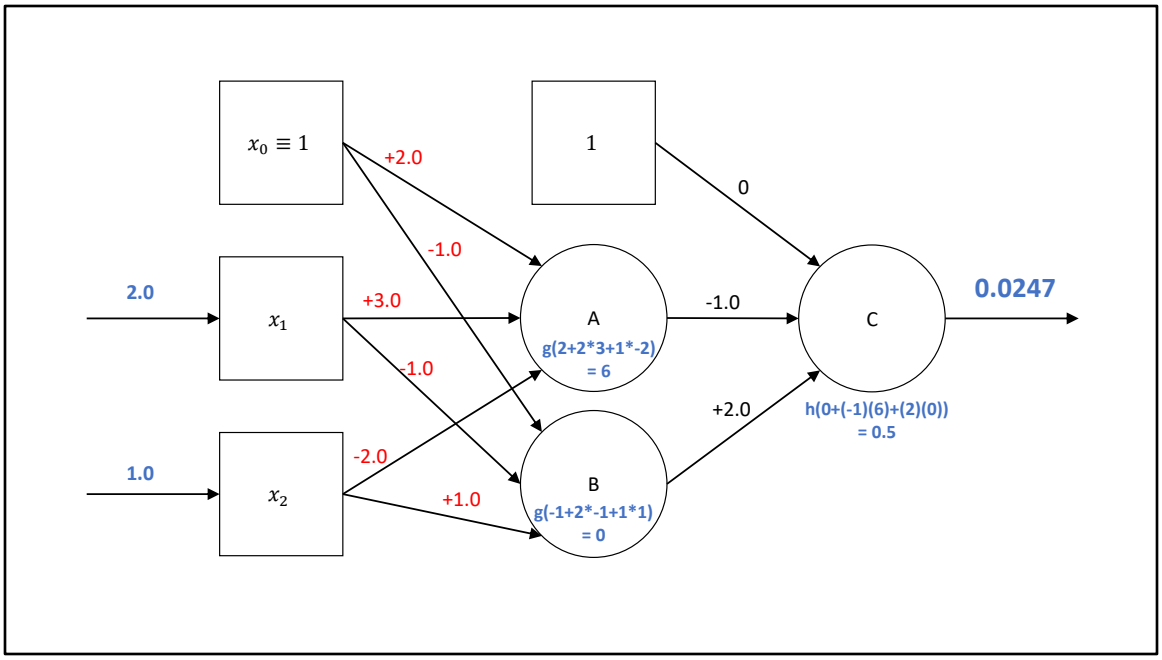
How do we use a neural network to make a prediction?

Using a Feedforward Network

- Suppose we have **learned** the network
- How do we use it to make a **prediction**?
- **Feed feature values** into the **input layer** (usually just $f(z) = z$)
- **Feed output of input layer** into **first hidden layer**
- Apply all units to compute unit output values
- **Feed** those outputs into the **next layer** (outputs become next inputs)
- Continue to feed forward through layer by layer
- Finally compute outputs of output layer (one neuron or many)


$x_0 \equiv 1$
1
+2.0
0
-1.0
2.0
x_1
+3.0
A
-1.0
C
?
-1.0
+2.0
1.0
x_2
-2.0
B
+1.0


$x_0 \equiv 1$
1
+2.0
0
-1.0
2.0
x_1
+3.0
A
g(2+2*3+1*-2)
= 6
-1.0
C
?
-1.0
+2.0
1.0
x_2
-2.0
B
g(-1+2*-1+1*1)
= 0
+1.0


$x_0 \equiv 1$
1
+2.0
0
-1.0
2.0
x_1
+3.0
A
g(2+2*3+1*-2)
= 6
-1.0
C
0.0247
-1.0
+2.0
h(0+(-1)(6)+(2)(0))
= 0.5
1.0
x_2
-2.0
B
g(-1+2*-1+1*1)
= 0
+1.0

Feedforward Networks

- No cycles
  - Note: that would be a recurrent neural network (RNN)
- Output of each layer becomes the input to the next layer
- The last layer (output from output layer) gives us the results

What makes it a deep network?

- Deep learning is just a multi-layer perceptron
- **Usually with many hidden layers**
- Potentially complex structure
- Leverages new choices for activation function (e.g., ReLU)
- Computational resources: many times faster, memory, parallel (GPUs)

Parameters and Hyperparameters

- What are the parameters?
  - Weights and biases for all the units
- What are the hyperparameters?
  - Structure of the network (number hidden layers, number units, etc.)
  - Choice of activation function
  - There are many others (we will see them in future lessons …)

Preview of Coming Attractions

What to expect the next few days

Preview

- How to **train** and **design** a network?
  - Backpropagation (finding the weights)
  - Hyperparameter selection
- Large Language Models (e.g., GPT-4o)

Tools

import tensorflow as tf

```
import tensorflow
```

```
import tensorflow as tf
```

Tools

- Don't use sklearn for real problems
- Go through tutorial (linked from Moodle): https://machinelearningmastery.com/tutorial-first-neural-network-python-keras/
- Training is computationally expensive: GPUs very useful (buy or rent)

Recap

What did we learn?

What did we learn?

- What a neural network is
- How each unit (neuron) works
- Activation function, why important, options
- How to use a feedforward network to get a prediction
- Tools: tensorflow and keras in Jupyter notebook

What are your questions?

- Visit during office hours, or contact me via email, etc.!